LUCAS TAM

+1(425) 362-4059 — lucasktam@gmail.com — linkedin.com/in/lucasktam — github.com/lucasktam

EDUCATION

University of Michigan Expected Graduation: May 2027
B.S. Engineering in Computer Science, GPA: 3.58 *Ann Arbor, MI*

Relevant Courses: Data Structures and Algorithms, Machine Learning, Operating Systems, Distributed Systems, Web Systems, Computer Organization, Computer Science Theory, Linear Algebra

EXPERIENCE

Software Development Engineer Intern May-Aug 2026
Amazon - Selling Partners Support Org *Seattle, WA*

- Created a system for creating version-controlled rubrics used by LLMs to score user-written documents, letting ~20 users across 3 teams define grading criteria used to evaluate content for dozens of downstream users.
- Built the service layer in Java, writing new REST API routes and rewriting all legacy endpoints; implemented atomic multi-item writes to keep revisioning and publishing consistent.
- Engineered data layer in DynamoDB (partition/sort-key schema, 2 GSIs) with sentinel "alias row" pattern (revisions 0/-1) optimizing latest- and published-revision reads to single lookups.

Software Development Engineer Intern May-Aug 2025
Amazon - Selling Partners Support Org *Seattle, WA*

- Built and launched a React dashboard used by ~50 PMs and engineers across 4+ teams to retrieve support cases, replacing manual AWS CloudWatch queries and saving ~3 hours/week per team.
- Designed a translation layer converting dashboard filter inputs into CloudWatch's structured query language, and shaped the returned data into a sortable, filterable case view with pagination.

PROJECTS

2D Game Engine | *C++, Lua*

- Built a component-based 2D game engine in C++ integrating third-party libraries (Box2D, SDL, RapidJSON) into a unified rendering, physics, audio, and input pipeline.
- Designed a Lua scripting bridge (via LuaBridge) exposing core engine systems to game logic, enabling data-driven gameplay without recompiling C++.

Multithreaded Network Fileserver | *C++, Boost Library, Threads, Sockets*

- Designed and implemented a concurrent, crash-consistent network file server in C++ that serves multiple simultaneous clients over a nested directory hierarchy via POSIX sockets.
- Implemented reader-writer locking with hand-over-hand traversal for safe concurrent access, and ensured crash consistency through ordered disk writes that maintain valid metadata invariants.

Thread Library | *C++, Unix*

- Built a preemptive, user-level thread library in C++ simulating multi-CPU scheduling, using `ucontext` for context switching with timer-driven preemption and FIFO-ordered ready queues.
- Implemented interrupt-safe synchronization primitives (mutex, condition variable) that disable and restore interrupts around critical sections to guarantee atomicity across the simulated CPUs.

TECHNICAL SKILLS

Programming Languages: C++, C, Python, Java, JavaScript, Lua, SQL, HTML, CSS

Programming Frameworks and Tools: React.js, DynamoDB, Linux/POSIX, Boost, Sockets, Pandas, Numpy, Concurrent Programming, Make, Git